

Reviewer Pack — Minimal Rank of Group Representations vs DPLL Search Tree He...

Entry 7d536530206d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 15:51:32 UTC

Minimal Rank of Group Representations vs DPLL Search Tree Height

Entry ID: 7d536530206d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 15:51:32 UTC

1. Conjecture

Field A (mathematical branch): Group Representation Theory **Field B** (complexity object): Complexity Theory: DPLL Search Tree Complexity

Statement:

[“For every k -CNF instance with n variables, the minimal rank of a linear representation of the monoid of boolean functions induced by the instance’s Tseitin matrix is $\Theta(\log(n) / \log(k))$ where k is the maximum clause size.”, ‘This minimal rank is equal to the height of the DPLL search tree for the given CNF instance.’, ‘For all instances with maximum clause size k , there exists a linear representation of the monoid that achieves this minimal rank.’]

Rationale (proposer’s reasoning):

[‘Group representation theory provides a framework for studying boolean functions through linear algebra. By examining the minimal rank of group representations, we might uncover structural properties of DPLL search trees that are not apparent in their traditional tree structure.’, ‘If there is a direct correspondence between the minimal rank and the height of the DPLL search tree, it could suggest new algorithms or heuristics for solving SAT problems by manipulating group representations.’]

Taxonomy category: GROUP_REPRESENTATION_THEORY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9c5e0cdb3138e550

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all k -CNF instances with n variables, the computed minimal rank of the linear representation of the monoid of boolean functions induced by the Tseitin matrix matches $\Theta(\log(n) / \log(k))$ within a margin of $\pm 10\%$ and the height of the DPLL search tree also matches this ratio within the same margin. The conjecture is

falsified if there exists any instance where the computed minimal rank or the DPLL search tree height does not match the expected ratio within the spec

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	1.00	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "group representation theory" AND "DPLL search tree height" - "linear representation" AND monoid boolean functions AND Tseitin matrix" - "minimal rank of group representations" AND CNF instance AND $\Theta(\log(n) / \log(k))$

Top relevant hits considered: - [http://arxiv.org/abs/2001.05888v1] Tropical linear representations of the Chinese monoid - [http://arxiv.org/abs/1701.06807v2] Linear Representation of Symmetric Games - [http://arxiv.org/abs/math/9806050v1] A Faithful Representation of the Singular Braid Monoid on Three Strands

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def tseitin_matrix(clauses):
        variables = set()
        new_vars = {}

        for clause in clauses:
            if len(clause) == 1:
                literals = clause
            else:
                literals = clause + [-x for x in clause]

            for literal in literals:
                if literal not in variables:
                    variables.add(literal)

        new_var = max(variables) + 1
        new_vars[clause] = new_var
```

```

        for i in range(len(clause)):
            for j in range(i + 1, len(clause)):
                matrix.append([new_var, -clause[i], -clause[j]])

    return matrix, variables

def gaussian_elimination(matrix):
    n = len(matrix)
    m = len(matrix[0])

    for i in range(n):
        # Find the pivot
        if matrix[i][i] == 0:
            for j in range(i + 1, n):
                if matrix[j][i] != 0:
                    matrix[i], matrix[j] = matrix[j], matrix[i]
                    break
            else:
                return None # Singular matrix

    # Eliminate above and below the pivot
    for i in range(n):
        for j in range(n):
            if i != j:
                factor = matrix[j][i] / matrix[i][i]
                for k in range(m):
                    matrix[j][k] -= factor * matrix[i][k]

    # Normalize the rows
    for i in range(n):
        pivot = matrix[i][i]
        for j in range(m):
            matrix[i][j] /= pivot

    return matrix

def rank(matrix):
    if not matrix:
        return 0

    n = len(matrix)
    m = len(matrix[0])

    row_rank = 0
    col_rank = 0

    for i in range(n):
        if all(matrix[i][j] == 0 for j in range(m)):
            continue

        # Find a non-zero column to pivot on
        pivot_col = -1
        for j in range(m):
            if matrix[i][j] != 0:
                pivot_col = j

```

```

        break

    # Swap columns to bring the pivot to the first position
    if pivot_col != 0:
        matrix[i][:], matrix[i][pivot_col:] = matrix[i][pivot_col:], matrix[i][:pivot_col]

    # Normalize the row
    for j in range(m):
        matrix[i][j] /= matrix[i][0]

    # Eliminate above and below the pivot
    for j in range(n):
        if i != j:
            factor = matrix[j][0]
            for k in range(m):
                matrix[j][k] -= factor * matrix[i][k]

    row_rank += 1

    return row_rank

def dpll(clauses, assignment):
    if not clauses:
        return True
    unit_clause = [c for c in clauses if len(c) == 1]
    if unit_clause:
        literal = unit_clause[0][0]
        new_assignment = assignment.copy()
        new_assignment[literal] = True
        if dpll([c for c in clauses if literal not in c], new_assignment):
            return True
        new_assignment[literal] = False
        if dpll([c for c in clauses if -literal not in c], new_assignment):
            return True
        return False

    literal = random.choice(clauses[0])
    new_assignment = assignment.copy()
    new_assignment[literal] = True
    if dpll([c for c in clauses if literal not in c], new_assignment):
        return True
    new_assignment[literal] = False
    if dpll([c for c in clauses if -literal not in c], new_assignment):
        return True
    return False

def height_dpll(clauses):
    assignment = {}
    return 1 + max(height_dpll(c) for c in clauses if literal in c)

n = random.randint(5, 40)
k = random.randint(2, min(n // 2, 10))
clauses = []
variables = set()

for _ in range(random.randint(3 * n, 6 * n)):

```

```

        clause_size = random.randint(1, k)
        literals = [random.choice([-i, i]) for i in range(1, n + 1)]
        if len(set(literals)) == clause_size:
            clauses.append(literals)

matrix, variables = tseitin_matrix(clauses)
rank_value = rank(gaussian_elimination(matrix))
height = height_dp11(clauses)

expected_rank = math.log(n) / math.log(k)
within_margin = abs(rank_value - expected_rank) <= 0.1 * expected_rank

return {
    "metric_name": "rank",
    "metric_value": rank_value,
    "instances_tested": len(clauses),
    "conjecture_holds": within_margin and height == expected_rank,
    "counterexample": "" if within_margin else f"Rank {rank_value} vs. expected {expected_rank}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    total_rank = 0
    count_support = 0

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        total_rank += result["metric_value"]
        if result["conjecture_holds"]:
            count_support += 1

    mean_rank = total_rank / len(seeds)
    support_fraction = count_support / len(seeds)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0 support_fraction={support_fraction}")
    elif any(result["conjecture_holds"] is False for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_783d4dbb.py", line 178, in <module>

```

    result = run_trial(seed)
    ^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_783d4dbb.py", line 154, in run_trial
    matrix, variables = tseitin_matrix(clauses)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_783d4dbb.py", line 42, in tseitin_matrix
    return matrix, variables
    ^^^^^
NameError: cannot access free variable 'matrix' where it is not associated with a value in enclosing scope

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14790
2	preregistration	ollama_remote	glm4:latest	0	0	10200
3	novelty	ollama_remote	glm4:latest	0	0	8419
4	novelty	ollama_remote	glm4:latest	0	0	9832
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16616
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15048
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12913
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16723
9	judge	ollama_remote	glm4:latest	0	0	16248

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 120788 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/7d536530206d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7d536530206d.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/7d536530206d.tar.gz` (if generated)